Local AI Coding Agent

Complete Architecture, Graph, Components, Security, Ops & Workflow Reference

A self-hosted, agentic software-engineering assistant built with LangGraph, a local LLM (via Ollama), retrieval-augmented code search, real tool execution, human-in-the-loop approval, and a self-correcting test loop — with PostgreSQL-backed persistence. This complete edition covers every production concern: agent state schema, memory extraction, RAG ingestion/update, tool schemas, security sandboxing, retry limits, error handling, authentication/authorization, observability, evaluation, configuration, git workflow, and deployment — plus a full-color LangGraph node/edge diagram.

Agent framework	LangGraph (explicit StateGraph, conditional edges, interrupt())
LLM runtime	Ollama (local, coding-specialized model)
Short-term memory	LangGraph Checkpointer — PostgreSQL
Long-term memory	LangGraph Store — PostgreSQL + memory extraction
Code retrieval	RAG over repo (ingestion + incremental update pipeline)
Vector DB	Chroma / FAISS / pgvector
Execution	File tools, sandboxed terminal, git tools — schema-validated
Safety gate	Allowlist + sandbox + timeout + HITL approval
Verification	Automated tests + bounded self-correction loop
Access control	FastAPI auth/authorization per user/project
Ops	Structured logging, evaluation benchmark, env-based config, CI/CD deployment

Contents

- 1. What This Project Is
- 2. Full System Architecture + Full-Color LangGraph Diagram
- 3. Agent State Schema
- 4. Core Components (Checkpointer, Store + Memory Extraction, RAG + Ingestion/Update)
- 5. Tools — Schemas & Selection Mechanism
- 6. Human-in-the-Loop & Security Hardening
- 7. Testing, Self-Correction, Retry Limits & Error Handling
- 8. Authentication & Authorization
- 9. Evaluation & Observability
- 10. PostgreSQL — Persistent Checkpointer & History
- 11. Configuration & Environment
- 12. Git Workflow — Branch, Diff, Approval, Commit/Rollback
- 13. Deployment — Local to Production
- 14. Recommended Free / Local Stack
- 15. Suggested Build Order (Roadmap)

1. What This Project Is

A local, privacy-preserving AI coding agent that behaves like a junior engineer sitting inside your repository. It reads and understands your codebase, remembers project context across sessions, retrieves only relevant files, runs terminal commands and tests, proposes and applies code changes, and asks for approval before doing anything risky.

- **Agentic loop** — the LLM decides which tool to call, not just what text to output.
- **Grounded in real code** — RAG retrieves only relevant files/functions via embeddings.
- **Persistent memory** — short-term + long-term memory, both durable via PostgreSQL.
- **Real execution** — runs shell commands, applies diffs, runs the test suite.
- **Layered safety** — allowlist, sandbox, timeout, and human approval before anything destructive.
- **Self-correction** — failing tests feed back into the LLM within a bounded retry limit.
- **Runs fully local** — Ollama + a local coding model, no API cost, no code leaves the machine.

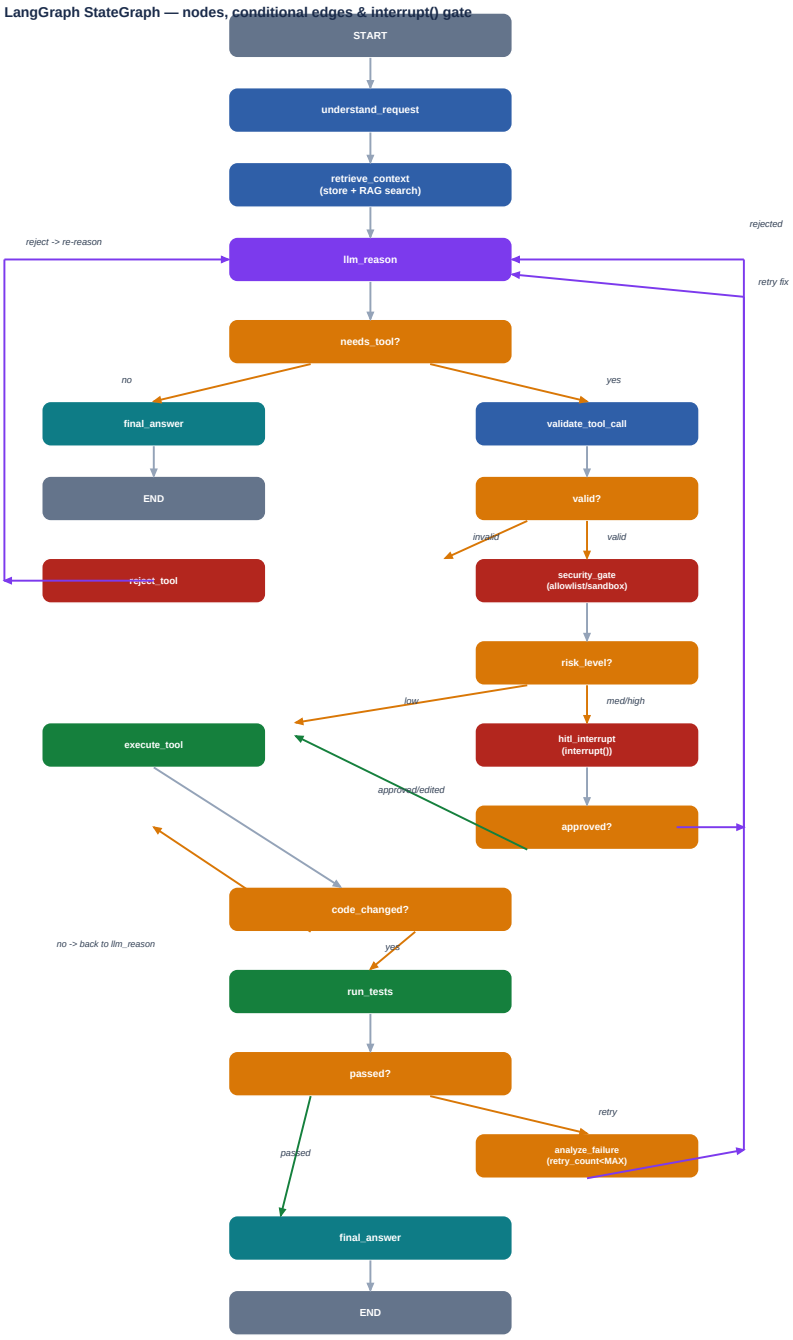
2. Full System Architecture

Text-form request path — user message, through FastAPI and the LangGraph agent, out to memory/retrieval, into the local LLM, through gated tool execution, and through the bounded test-and-fix loop.

```
USER / UI (Streamlit) -> FastAPI (auth/authz, routing) -> LangGraph Agent
   -> [Checkpointer | LTM Store | RAG Retrieval] -> Local LLM (Ollama)
   -> Tool Decision -> Security Gate (allowlist/sandbox/timeout) -> HITL Approval
   -> Execute -> Run Tests -> PASS -> Final Answer
                          -> FAIL -> Analyze -> Fix again (bounded) -> Run Tests
```

2.1 Full-Color LangGraph Node/Edge Diagram

This is the actual compiled StateGraph: every node is a Python function operating on AgentState, every diamond-colored box is a conditional edge (router function), and the purple lines are loop-back edges that return control to **llm_reason**.



Blue = process nodes, purple = LLM reasoning, amber = conditional/router edges, red = security & HITL gates, green = execution/tests, teal = terminal answer, gray = start/end. Purple lines trace every path that loops back into llm_reason.

3. Agent State Schema

LangGraph passes a single typed **state object** between every node. This is what gets persisted by the Checkpointer at each step.

```
from typing import TypedDict, Literal, Optional
from langgraph.graph.message import add_messages
from typing_extensions import Annotated

class AgentState(TypedDict):
    messages: Annotated[list, add_messages]       # full chat history (STM)
    retrieved_code: list[dict]                      # [{file, chunk, score, start_line, end_line}
]
    retrieved_memory: list[dict]                    # facts pulled from LTM store
    pending_tool_call: Optional[dict]                # {name, args} chosen by the LLM
    tool_result: Optional[dict]                      # {name, output, success, error}
    test_result: Optional[dict]                       # {passed, failed_count, stdout, stderr}
    retry_count: int                                   # bounded by MAX_RETRIES
    approval_status: Literal["pending","approved","rejected","edited", None]
    risk_level: Literal["low", "medium", "high"]
    thread_id: str
    project_namespace: str
    user_id: str                                       # for authz checks (Section 8)
```

4. Core Components

4.1 LangGraph — Orchestration

Controls the entire workflow as an explicit state graph. Each node is a discrete step operating on AgentState; edges define what happens next based on the LLM's decision.

4.2 Ollama + Local Coding LLM

The reasoning engine — runs fully on local hardware, no API cost, no code leaves the machine. A coding-specialized model (Code Llama / Qwen-Coder / DeepSeek-Coder class) is recommended for accurate diffs.

4.3 Checkpointer — Short-Term Memory (STM)

Tracks the state of the current run, keyed by thread_id — lets the agent resume exactly where it left off, including after a HITL pause.

4.4 Store — Long-Term Memory (LTM)

Durable, cross-session facts (project name, stack, conventions, preferences), keyed by namespace, retrieved selectively per query.

```
namespace = ("user", "123")
project -> "ANPR project"     stack -> "Python + YOLO"     preference -> "concise answers"
store.search(namespace, query=current_question, limit=3)
```

4.4.1 Memory Extraction

Not every message should be saved — an extraction step runs after each turn to decide what's actually durable and reusable.

```
EXTRACTION_PROMPT = """
Extract only durable, reusable facts (stack choices, conventions, stated
preferences). Ignore one-off questions and small talk.
Return JSON: [{"key": str, "value": str, "confidence": float}]
"""
extracted = llm.invoke(EXTRACTION_PROMPT + latest_turn)
for fact in extracted:
    if fact["confidence"] >= 0.7:
        store.put(namespace, fact["key"], fact["value"])   # dedup on key
```

4.5 RAG / Code Retrieval

The repo is chunked, embedded, and stored in a vector DB; only the most relevant chunks are retrieved per query — never the whole repository.

4.5.1 RAG Ingestion Pipeline

```
Repo root -> 1. File filter (include code, exclude node_modules/.git/build/binaries)
          -> 2. Chunking (AST-aware by function/class, or ~200-400 token window + overlap)
          -> 3. Metadata (file_path, start_line, end_line, git_commit_hash, symbol_name)
          -> 4. Embeddings (local embedding model, e.g. nomic-embed-text via Ollama)
          -> 5. Vector DB upsert (Chroma/FAISS/pgvector, id = hash(path + chunk_index))
```

4.5.2 RAG Update — keeping the index in sync

```
def sync_index(changed_files, deleted_files):
    for path in deleted_files:
        vector_db.delete(where={"file_path": path})
    for path in changed_files:
        vector_db.delete(where={"file_path": path})        # drop stale chunks
        chunks = chunk_file(path)
        vector_db.upsert(ids=[...], embeddings=embed(chunks), metadatas=[...])
# Triggered by git post-commit hook / file watcher / pre-run check
```

- Delete-then-reinsert per file is simpler and safer than diffing individual chunks.
- Store commit hash in metadata to skip no-op re-runs.

5. Tools — Schemas & Selection Mechanism

Tool	Purpose
`read_file()`	Read a file's contents for context
`write_file()`	Apply an edit / create a file (HITL gated)
`search_code()`	Semantic/keyword search across the index
`run_command()`	Execute a shell command — sandboxed
`run_tests()`	Run the test suite, capture pass/fail
`git_diff()/git_status()`	Inspect pending changes

5.1 Tool schema exposed to the LLM

```
{
  "name": "read_file",
  "description": "Read the contents of a file inside the project workspace.",
  "parameters": {"type": "object",
    "properties": {"path": {"type": "string", "description": "Relative path from repo root"}},
    "required": ["path"]}
}
# Every tool returns this result contract:
{"tool_name": "read_file", "success": true, "output": "...", "error": null, "duration_ms": 42}
```

5.2 Selection-to-execution mechanism

```
LLM: "I need auth.py" -> Tool selection -> read_file(path="auth.py")
   -> Schema validation (reject bad args / out-of-workspace paths)
   -> Security gate (allowlist + sandbox + timeout, Section 6)
   -> Execute -> Result -> AgentState.tool_result -> back to LLM
```

5.3 Terminal / CMD Execution

Real execution against the actual project, not simulated output — this is how the agent verifies its own reasoning.

```
Agent -> run_command("pytest") -> Terminal executes -> Output: "2 tests failed"
      -> LLM analyzes the failure -> Generates a fix
```

6. Human-in-the-Loop & Security Hardening

Before any risky operation the agent pauses via LangGraph's **interrupt()**, which suspends execution and persists state until a decision arrives.

```
LLM: "I want to modify auth.py."  ->  HITL: "Apply this change?"  [Approve] [Reject] [Edit]
  -> approval -> write_file()  or  run_command()
```

Why HITL matters:

- Prevents destructive edits or accidental command execution from running unattended.
- Because state is checkpointed, the pause can last indefinitely (even across app restarts) — the graph resumes exactly where it stopped once you respond.
- The 'Edit' path lets you modify the proposed diff/command before it runs, not just accept or reject outright.

LLM → run_command("rm -rf /") → must be BLOCKED before it ever reaches a human approval prompt. A rushed approval, an automation pre-approving routine calls, or a prompt-injection from retrieved code could otherwise make HITL the only defense — which is not enough alone.

Layer	How it works
`Command allowlist`	run_command() checks the binary against a fixed list before HITL is even shown
`Sandbox`	Execution in a restricted container/subprocess, no network, non-root user
`Timeout`	Every subprocess wrapped with a hard timeout; exceeding it kills the process
`Workspace/path restriction`	File tools validate paths resolve inside the repo root
`HITL approval`	Final human checkpoint for anything still classified medium/high risk

```
ALLOWED_COMMANDS = {"pytest", "python", "npm", "git", "ls", "cat", "grep"}
def run_command(cmd: str, timeout: int = 60):
    if cmd.split()[0] not in ALLOWED_COMMANDS:
        return {"success": False, "error": "not on allowlist"}
    try:
        r = subprocess.run(cmd, shell=False, cwd=WORKSPACE_ROOT, timeout=timeout,
                           capture_output=True, text=True, env={"PATH": "/usr/bin"})
        return {"success": r.returncode == 0, "output": r.stdout, "error": r.stderr}
    except subprocess.TimeoutExpired:
        return {"success": False, "error": f"exceeded {timeout}s timeout"}
```

7. Testing, Retry Limits & Error Handling

```
Generate fix -> Apply fix -> Run tests -> PASS -> Finish
                                       -> FAIL -> retry_count += 1
                                                -> < MAX_RETRIES? Analyze -> Fix again
                                                -> else: Stop, report failure to user
```

- **MAX_RETRIES** (e.g. 3) enforced in the graph's conditional edge, not left to the LLM.
- On hitting the cap, stop and report — never loop silently forever.
- Each retry should re-run search_code() if the error suggests missing context.

Failure type	Handling
Tool execution fails	Return {success:false, error} to LLM; don't crash the graph
LLM call fails/times out	Retry with backoff (2x); else surface clear error to user
Command timeout	Kill process, return timeout error, don't silently retry same cmd
Invalid/hallucinated tool call	Reject at schema-validation stage before execution
Test suite crashes (not just fails)	Treat distinctly from FAIL; surface stack trace
Approval times out	Leave graph interrupted indefinitely; never auto-approve

8. Authentication & Authorization

The FastAPI layer must know *who* is making a request and *what they're allowed to touch* — without this, any user could act on any project/repo through the agent.

8.1 Authentication (who is this?)

- JWT or session-based auth on every FastAPI endpoint; reject unauthenticated requests before they reach LangGraph.
- Attach user_id to the AgentState at the very start of the graph (understand_request), so it flows through every node and every log entry.

8.2 Authorization (what can they do?)

- **Project-level access control:** a user can only invoke the agent against repos/projects they're a member of — checked before project_namespace is set.
- **Role-based tool permissions:** e.g. a "viewer" role can call read_file/search_code but not write_file/run_command; enforce this in the security gate, not just the UI.
- **Per-project HITL approvers:** only users with an "approver" role for that project can resolve a pending interrupt() — validate this server-side when the approval endpoint is hit.

```
@router.post("/agent/invoke")
async def invoke_agent(req: AgentRequest, user: User = Depends(get_current_user)):
    if not has_project_access(user.id, req.project_id):
        raise HTTPException(403, "not authorized for this project")
    config = {"configurable": {"thread_id": req.thread_id}}
    state = {"user_id": user.id, "project_namespace": req.project_id, ...}
    return graph.invoke(state, config=config)

@router.post("/agent/approve")
async def approve(req: ApprovalRequest, user: User = Depends(get_current_user)):
    if not has_approver_role(user.id, req.project_id):
        raise HTTPException(403, "not authorized to approve changes")
    ...
```

9. Evaluation & Observability

9.1 Observability — what to log for every run

```
{"thread_id": "ABC123", "node": "tool_execution", "timestamp": "2026-09-02T10:14:02Z",
 "input": {"tool": "read_file", "args": {"path": "auth.py"}},
 "output": {"success": true, "duration_ms": 38}, "llm_latency_ms": 812,
 "retry_count": 0, "risk_level": "low", "user_id": "u_42"}
```

- What the agent reasoned/decided at each step, not just the final answer.
- Every tool call: name, arguments, output, success/failure, duration.
- Test results per attempt and retries used.
- Latency per LLM call and per tool call.
- Every HITL decision and who made it (ties into Postgres audit trail, Section 10).

9.2 Evaluation — measuring whether it actually works

- **Task set:** fixed list of realistic tasks with known-correct outcomes.
- **Success metric:** % of tasks where tests pass after the fix, within retry limit.
- **Retrieval quality:** did search_code() surface the relevant file (precision/recall)?
- **Safety metric:** % of dangerous test prompts correctly blocked before HITL.
- **Efficiency:** average retries-to-pass, average end-to-end latency.
- Re-run after every model/prompt/retrieval change to catch regressions early.

10. PostgreSQL — Persistent Checkpointer & History

Both the Checkpointer (STM) and Store (LTM) should be backed by PostgreSQL — durable history that survives restarts and can be audited.

Table	Contents
`checkpoints`	Full graph state snapshot per thread_id at each step
`checkpoint_writes`	Pending writes between checkpoints — exact mid-step resume
`store (LTM)`	Namespaced long-term facts from memory extraction
`tool_call_log`	Audit trail: input, output, approver, timestamp, risk_level
`embeddings (pgvector, optional)`	Chunked code/doc vectors for RAG

```
from langgraph.checkpoint.postgres import PostgresSaver
from langgraph.store.postgres import PostgresStore
DB_URI = "postgresql://user:password@localhost:5432/agent_db"
with PostgresSaver.from_conn_string(DB_URI) as checkpointer:
    checkpointer.setup()
    graph = builder.compile(checkpointer=checkpointer, store=store)
```

11. Configuration & Environment

Nothing environment-specific should be hardcoded — model name, DB URL, repo path, retry count, and allowed commands all belong in config/env, loaded once at startup.

```
# .env
OLLAMA_MODEL=qwen2.5-coder:14b
OLLAMA_BASE_URL=http://localhost:11434
DATABASE_URL=postgresql://user:password@localhost:5432/agent_db
REPO_WORKSPACE_ROOT=/workspace/project
VECTOR_DB=chroma            # chroma | faiss | pgvector
EMBEDDING_MODEL=nomic-embed-text
MAX_RETRIES=3
COMMAND_TIMEOUT_SECONDS=60
ALLOWED_COMMANDS=pytest,python,npm,git,ls,cat,grep
JWT_SECRET=change_me
LOG_LEVEL=INFO
```

```
# config.py
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    ollama_model: str
    database_url: str
    repo_workspace_root: str
    max_retries: int = 3
    command_timeout_seconds: int = 60
    allowed_commands: list[str]
    class Config:
        env_file = ".env"

settings = Settings()   # single source of truth, imported everywhere
```

- Never commit .env — provide .env.example with placeholder values.
- Separate .env.dev / .env.prod; inject prod secrets via the deployment platform, not the repo.

12. Git Workflow — Branch, Diff, Approval, Commit/Rollback

The agent should never write directly to a shared branch. Every change goes through an isolated branch, a visible diff, human approval, and only then a commit — with rollback always available.

```
1. git_branch_create  ->  agent creates/switches to "agent/<thread_id>-<short-desc>"
2. Apply proposed edit inside that branch (write_file, still HITL-gated)
3. git_diff            ->  full diff shown to the user alongside the HITL prompt
4. HITL approval        ->  Approve / Reject / Edit  (Section 6)
5a. Approved  -> git_commit  ("agent: <summary>", co-authored-by: agent)
5b. Rejected  -> git_checkout -- <files>   (discard the working-tree change)
6. Merge      -> opened as a PR for a human to merge, OR auto-merge only if
                 explicitly configured for low-risk changes
7. Rollback   -> git revert <commit_sha>   if a merged agent change breaks something later
```

- One branch per agent thread keeps runs isolated and makes the diff reviewable in one place.
- Commits are always attributed to the agent (not the human approver) for a clean audit trail — the approver is recorded separately in tool_call_log.
- Prefer PRs over direct pushes even for auto-approved low-risk changes, so there's always a revert point.

13. Deployment — Local Development to Production

Stage	Setup
`Local dev`	docker-compose: FastAPI + Postgres + Ollama + Streamlit, all on localhost, .env.dev
`Staging`	Same containers on a small VM/cloud instance; seeded test repo; .env.staging with real Postgres
`Production`	Containers behind a reverse proxy (nginx/Caddy) with TLS; managed Postgres (backups + point-in-time re
`CI/CD`	On push: run lint + unit tests + the evaluation benchmark (Section 9.2); block merge on regression; build &
`Secrets`	Injected via the platform's secret manager (not baked into images or committed .env files)
`Scaling`	Stateless FastAPI replicas behind a load balancer; Postgres and the vector DB are the only stateful pieces

```
# docker-compose.yml (simplified)
services:
  api:      { build: ./backend, env_file: .env, depends_on: [db, ollama] }
  ui:       { build: ./frontend, ports: ["8501:8501"] }
  db:       { image: pgvector/pgvector:pg16, volumes: ["pgdata:/var/lib/postgresql/data"] }
  ollama:   { image: ollama/ollama, volumes: ["ollama_models:/root/.ollama"] }
volumes: { pgdata: {}, ollama_models: {} }
```

- Run the RAG ingestion pipeline (Section 4.5.1) as a one-off job on first deploy, then the update pipeline continuously via a webhook/worker.
- Health checks on /health for the API and a readiness check that confirms Ollama has the model pulled before accepting traffic.

14. Recommended Free / Local Stack

Layer	Choice
Frontend	Streamlit
Backend	FastAPI (+ auth/authorization middleware)
Agent orchestration	LangGraph (explicit AgentState schema)
LLM runtime	Ollama
Coding model	Local coding-specialized model (Ollama)
Short-term memory (STM)	LangGraph Checkpointer — PostgreSQL
Long-term memory (LTM)	LangGraph Store — PostgreSQL + memory-extraction step
Embeddings	Local embedding model
Vector DB	Chroma / FAISS (or pgvector inside Postgres)
Tools	Python functions with strict JSON schemas
Terminal execution	subprocess, sandboxed, allowlisted, timeout-bound
HITL	LangGraph interrupt()
Config	Pydantic Settings via .env
Observability	Structured JSON logs per node, keyed by thread_id
CI/CD	GitHub Actions: lint + tests + eval benchmark -> Docker build

15. Suggested Build Order (Roadmap)

- **1 — Skeleton:** FastAPI + Streamlit + a single LangGraph node with AgentState defined up front.
- **2 — Persistence:** PostgresSaver checkpointer; confirm resume across restarts.
- **3 — Long-term memory:** PostgresStore + memory-extraction step.
- **4 — RAG:** ingestion pipeline + update/sync pipeline.
- **5 — Tools:** schemas + result contracts for all six tools.
- **6 — Security:** allowlist, sandbox, timeouts, workspace restriction.
- **7 — HITL:** interrupt() + Approve/Reject/Edit UI.
- **8 — Self-correction:** test loop capped at MAX_RETRIES.
- **9 — Auth/authz:** per-user, per-project access control on every endpoint.
- **10 — Git workflow:** branch-per-thread, diff review, commit/rollback.
- **11 — Observability & audit:** structured logs + Postgres tool_call_log + UI history view.
- **12 — Evaluation:** benchmark task set, run on every significant change.
- **13 — Deployment:** docker-compose locally -> CI/CD -> managed Postgres + reverse proxy in production.

This document is a build reference, not a guarantee of behavior — validate the HITL gate, the security sandbox, and the test-retry cap with real usage before unattended or production use.

10A. Embedding, LTM & Database Storage — Deep Dive

This section explains exactly **how embeddings are created** for Long-Term Memory (LTM), how the **key-value argument data** is stored, and whether they share the **same database**.

■ **Answer:** Yes — both LTM embeddings and key-value argument data are stored in the **same PostgreSQL database** (running in Docker), but in **different tables/collections**.

10A.1 Three Storage Layers Inside One PostgreSQL Database

#	What Is Stored	Where (Table / Collection)	Module	Data Format
1	Chat History (STM)	checkpoints table	PostgresSaver (connection.py)	Binary serialized messages
2	Key-Value Arguments (key=SPORTS, value=CRICKET)	store table	PostgresStore (connection.py)	JSON {"value": "CRICKET"}
3	Semantic Embeddings (LTM vectors)	user_memories collection (pgvector)	PGVector (vector_db.py)	2560-dim float vectors

10A.2 Dual-Write Flow — What Happens When a Memory Is Saved

When the user (or the LLM via the **save_memory** tool) stores a memory like `key="SPORTS", value="CRICKET"`, two parallel writes happen:

```
User saves: key="SPORTS", value="CRICKET"
■
■■■■ 1. store.put(namespace, key, {"value": value})
■ ■→ PostgresStore → "store" table → Plain JSON (for direct key lookup)
■
■■■■ 2. save_semantic_memory(key, value)
■→ PGVector → "user_memories" collection → Embedding vector (for semantic
search)
```

Relevant code in **backend/graph/tools.py** (lines 249-250):

```
store.put(USER_NAMESPACE, key, {"value": value}) # ← Key-Value store (direct
lookup)
save_semantic_memory(key, value, user_id="1") # ← Vector embedding (semantic
search)
```

Why two writes?

- **store table** — when the tool directly calls `get_memory("SPORTS")`, the value is retrieved instantly by exact key lookup (fast, O(1)).
- **user_memories pgvector collection** — when the user asks *"What sport do I like?"*, cosine similarity search automatically finds **SPORTS: CRICKET** even though the user never mentioned the

exact key name (semantic search).

10B. All PostgreSQL Table Names — Complete Reference

Below are the **6 actual tables** created inside your PostgreSQL database, who creates them, and what they contain.

#	Table Name	Purpose	Created By	Source Module
1	checkpoints	Full graph state snapshot per thread_id at each step (STM)	PostgresSaver .setup()	backend/db/connection.py
2	checkpoint_blobs	Serialized binary message data for each checkpoint	PostgresSaver .setup()	backend/db/connection.py
3	checkpoint_writes	Pending writes between checkpoints — enables exact mid-step resume	PostgresSaver .setup()	backend/db/connection.py
4	store	Key-value LTM storage — (namespace, key) → JSON value e.g. SPORTS → {"value": "CRICKET"}	PostgresStore. setup()	backend/db/connection.py
5	langchain_pg_collection	Collection metadata — contains entries for "user_memories" and "codebase" collections	PGVector()	backend/rag/vector_db.py
6	langchain_pg_embedding	All embedding vectors (2560-dim floats) — both LTM and RAG vectors, filtered by collection_id FK	PGVector()	backend/rag/vector_db.py

10B.1 Database Table Tree

```
PostgreSQL Database (Docker — pgvector/pgvector:pg16)
■
■■■ checkpoints ← STM (chat states per thread_id)
■■■ checkpoint_blobs ← STM (serialized message data)
■■■ checkpoint_writes ← STM (write log for mid-step resume)
■
■■■ store ← LTM Key-Value (direct key lookup)
■
■■■ langchain_pg_collection ← Collection metadata
■ ■■■ id: uuid → "user_memories" (LTM semantic vectors)
■ ■■■ id: uuid → "codebase" (RAG code vectors)
■
■■■ langchain_pg_embedding ← All vectors (2560-dim floats)
■■■ collection_id FK → user_memories (LTM)
■■■ collection_id FK → codebase (RAG)
```

10B.2 Semantic Cosine Distance Filtering — Example

When a user sends a message, the chatbot node embeds the query and performs a **cosine distance search** against **user_memories** in pgvector. Only results with distance < 0.50 are injected into the system prompt:

User Query	Database Memory	Cosine Distance	Action
"What sport do I like?"	SPORTS: CRICKET	0.4384	■ Injected into prompt!
"What sport do I like?"	evening_food: momos	0.8210	■ Rejected (Irrelevant)
"What sport do I like?"	CODING LANGUAGE: PYTHON	0.7950	■ Rejected (Irrelevant)
"How to write quicksort?"	SPORTS: CRICKET	0.6120	■ Rejected (Prompt stays clean)

■ **Key Insight:** Even with 1000+ memories in the database, only 1-2 semantically relevant memories are ever loaded into the prompt. This makes LTM **immune to prompt bloat** — the system scales safely regardless of how many memories accumulate over time.